



Institute of Engineering and Technology (IET)

Major Project Synopsis (PR1107)

**Design and Verilog HDL Implementation of a
Controller Area Network (CAN) Protocol Controller
with FPGA/Embedded Hardware Realization**

PREPARED BY

Prakhar Agrawal (2023BTech057)

Pratham Bansal (2023BTech060)

FACULTY GUIDE

Dr. Devika Kataria

August, 2026

Contents

1	Abstract	2
2	Introduction	2
3	Problem Statement	3
4	Objectives	3
5	Scope of Work	4
6	Methodology	4
6.1	Phase 1: Study and Analysis of CAN	4
6.2	Phase 2: Verilog-Based CAN Controller Design	4
6.3	Phase 3: Simulation and Verification	5
6.4	Phase 4: Hardware Implementation	5
7	Simulation, Verification & Reliability	6
8	Hardware and Software Requirements	7
9	Tentative Timeline	8
10	Expected Outcomes	8
11	Applications (Illustrative, Optional Extension)	9
12	References	10

1. Abstract

Controller Area Network (CAN) is a high-integrity serial communication protocol, standardized as ISO 11898, originally developed by Bosch to reduce point-to-point wiring complexity in vehicles and now widely used across automotive, industrial, railway, aerospace, and medical systems. Most embedded projects that use CAN treat it as an off-the-shelf peripheral, configuring an existing controller IP block without engaging with its internal logic. This project instead studies the internal operation of CAN and designs a functional CAN controller from scratch using Verilog HDL. The controller is decomposed into transmit and receive controllers, bit-level arbitration logic, bit stuffing/de-stuffing logic, CRC generation and checking, and ACK/error-detection logic, each implemented as RTL and driven by finite-state machines. The design is verified through Verilog testbenches simulating multiple CAN nodes on a shared bus, checking frame generation, reception, arbitration outcomes, bit stuffing, CRC correctness, ACK behaviour, and basic error detection at the signal and clock-cycle level. Once simulation is verified, the design is targeted to an FPGA/embedded development board and interfaced with a CAN transceiver to establish a physical CAN bus, with multiple boards subsequently configured as CAN nodes to demonstrate real inter-node communication. The project therefore builds CAN understanding at the RTL level rather than the peripheral-configuration level, and delivers a reusable, from-scratch CAN controller design.

2. Introduction

Controller Area Network (CAN) is a high-integrity serial communication protocol designed for reliable communication between intelligent electronic devices. It was originally developed by Bosch in 1985 to reduce the complexity, weight, and cost of point-to-point wiring in vehicles. CAN was later standardized as ISO 11898 and has become widely used in automotive, industrial, railway, aerospace, and medical applications.

CAN allows multiple nodes to communicate over a shared two-wire bus using a broadcast communication mechanism: every node observes every transmitted message and independently determines whether that message is relevant to it. One of the key features of CAN is its priority-based, non-destructive arbitration, which allows the highest-priority message to gain access to the bus without corrupting the ongoing transmission of any lower-priority contender. CAN also incorporates error-detection mechanisms such as cyclic redundancy check (CRC) and acknowledgement to provide reliable communication over a shared, potentially noisy medium.

This project aims to study the internal operation of the CAN protocol and design a functional CAN controller from scratch using Verilog HDL, rather than relying on a pre-built CAN peripheral. The designed controller will first be verified through simulation and subsequently targeted toward implementation on suitable FPGA/embedded hardware.

3. Problem Statement

Most embedded implementations of CAN use a microcontroller's built-in CAN peripheral or an external CAN controller IC, where the internal logic of frame construction, arbitration, bit stuffing, CRC, and error handling is already implemented in fixed silicon and hidden behind a register-level API. This gives working systems quickly, but it leaves the underlying digital design of the protocol – the exact hardware mechanism by which arbitration, error detection, and fault confinement are actually realised in logic – unexamined and unverified by the person using it. There is a gap between knowing how to configure a CAN peripheral and knowing how to design one: understanding CAN at the RTL level requires building the frame-construction and frame-decoding state machines, the bit-level arbitration comparator, the stuffing/de-stuffing logic, and the CRC and ACK logic as digital hardware, and verifying that this hardware reproduces the protocol's guarantees under simulation before trusting it on a bus with real nodes. This project addresses that gap by designing and verifying a CAN controller from first principles in Verilog HDL rather than treating CAN as a black-box peripheral.

4. Objectives

- Study the architecture and working principles of the CAN protocol, including its physical and data-link layer concepts relevant to a digital (RTL) implementation.
- Design the major functional blocks of a CAN controller using Verilog HDL: transmit controller, receive controller, arbitration logic, bit stuffing/de-stuffing logic, CRC logic, and ACK/error-detection logic.
- Implement CAN frame generation and reception in RTL, including the 11-bit standard arbitration identifier, control field, data field, CRC field, ACK field, and the associated bit-level sequencing.
- Implement bit-level, non-destructive arbitration so that a node transmitting a recessive bit while a dominant bit is observed on the bus correctly loses arbitration and stops transmitting.
- Implement bit stuffing and de-stuffing, CRC generation and verification, ACK handling, and basic error detection (CRC mismatch, stuffing errors, invalid frame behaviour).
- Verify the designed CAN controller using Verilog testbenches, including simulation of multiple CAN nodes communicating over a common simulated bus.
- Target the verified design to an FPGA/embedded development board, interface it with a CAN transceiver, and demonstrate communication between multiple hardware CAN nodes on a physical bus.

5. Scope of Work

The scope of this project covers the RTL design, in Verilog HDL, of a classical CAN controller supporting the standard CAN data frame with an 11-bit arbitration identifier and a data field of up to 8 bytes, together with its verification through simulation and, subsequently, its deployment on FPGA/embedded hardware with a CAN transceiver to form a small multi-node bus (three or more boards). It includes frame generation and reception, bit-level arbitration, bit stuffing/de-stuffing, CRC generation and checking, ACK handling, and basic error detection, all implemented and verified as digital logic rather than configured on an existing CAN peripheral. It does not include the 29-bit extended identifier format, CAN-FD, higher-layer protocols built on top of CAN (such as CANopen, J1939, or UDS), custom transceiver or PCB design, or synthesis timing closure/STA on the final netlist, though these are noted as natural extensions once the base controller is verified.

6. Methodology

The project will be developed in stages, beginning with protocol analysis and progressing toward RTL implementation, simulation-based verification, and hardware deployment.

6.1 Phase 1: Study and Analysis of CAN

The first stage focuses on understanding the working of CAN and its communication architecture: the CAN bus, CAN nodes, dominant and recessive logic levels, message priorities, arbitration, error detection, and the structure of CAN frames. The initial implementation will focus on the standard CAN data frame, structured as Start-of-Frame (SOF), Arbitration ID, Control Field, Data Field, CRC, ACK, and End-of-Frame (EOF), in that sequence. The standard CAN frame uses an 11-bit arbitration identifier (CAN also supports an extended 29-bit identifier, out of scope here), and the data field can carry up to 8 bytes, with the DLC field indicating the amount of data carried by the frame. Figure 1 illustrates this field-level layout.

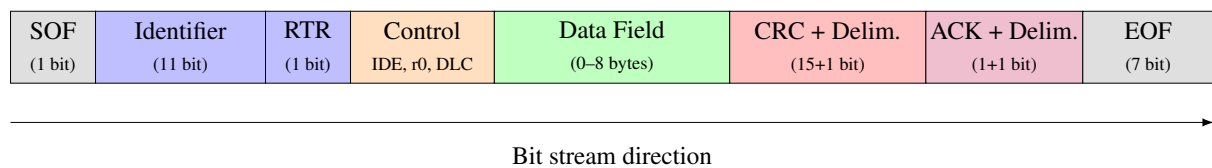


Figure 1: Field-level structure of a standard CAN data frame (11-bit identifier).

6.2 Phase 2: Verilog-Based CAN Controller Design

The core of the project is the RTL design of a CAN controller in Verilog HDL, divided into smaller hardware modules to simplify development and verification: a transmit controller, a receive controller, arbitration logic, bit stuffing logic, CRC logic, ACK handling, and error detection, all interfacing with the CAN bus through a common bus interface.

- **Transmit Controller:** accepts a message and constructs the corresponding CAN frame, generating the SOF, arbitration field, control field, data field, CRC field, ACK-related behaviour, and EOF in the correct sequence, driven by a finite-state machine.
- **Receive Controller:** monitors the CAN bus, detects the start of a frame, samples incoming bits, reconstructs the frame, removes stuffed bits, verifies the CRC, and extracts the received identifier and data.
- **Arbitration Logic:** implements bit-level, non-destructive arbitration – a node transmitting a recessive bit while detecting a dominant bit recognises it has lost arbitration and stops transmitting, allowing the higher-priority frame (lower CAN identifier) to continue without corruption.
- **Bit Stuffing / De-Stuffing:** the transmitter inserts a complementary bit after the required run of consecutive identical bits; the receiver performs the reverse operation to recover the original bitstream.
- **CRC Generation and Verification:** the transmitter generates the CRC associated with the frame, and the receiver independently calculates and compares the CRC to determine whether the received frame is valid.
- **ACK and Error Detection:** implements the acknowledgement mechanism, where a receiving node indicates successful frame reception through the ACK field, and investigates basic error conditions such as CRC mismatch, stuffing errors, and invalid frame behaviour.

Figure 2 shows the internal block-level architecture that this RTL design targets – a bit-stream-processor style organization (of the kind used in industry CAN controllers such as the SJA1000/MCP2515 family) comprising the bit-timing/synchronization logic, the stuffing and de-stuffing datapath, the CRC generation/check datapath, the transmit/receive error counters, and the central protocol finite-state machine that sequences all of these blocks. This architecture is used as the reference floor-plan for partitioning the Verilog modules described above.

6.3 Phase 3: Simulation and Verification

After the individual modules are developed, they are integrated into a complete CAN controller and verified using Verilog testbenches. The simulation environment represents multiple nodes connected to a shared CAN bus, and the testbench is used to verify correct CAN frame generation and reception, arbitration between multiple nodes, bit stuffing and de-stuffing, CRC generation and checking, ACK behaviour, basic error detection, and successful communication between multiple simulated nodes. Waveform analysis is used to verify the behaviour of individual signals and FSM states at the bit and clock-cycle level.

6.4 Phase 4: Hardware Implementation

Once the Verilog design has been successfully simulated and verified, the project moves toward hardware implementation. The CAN controller is targeted to a suitable FPGA/embedded devel-

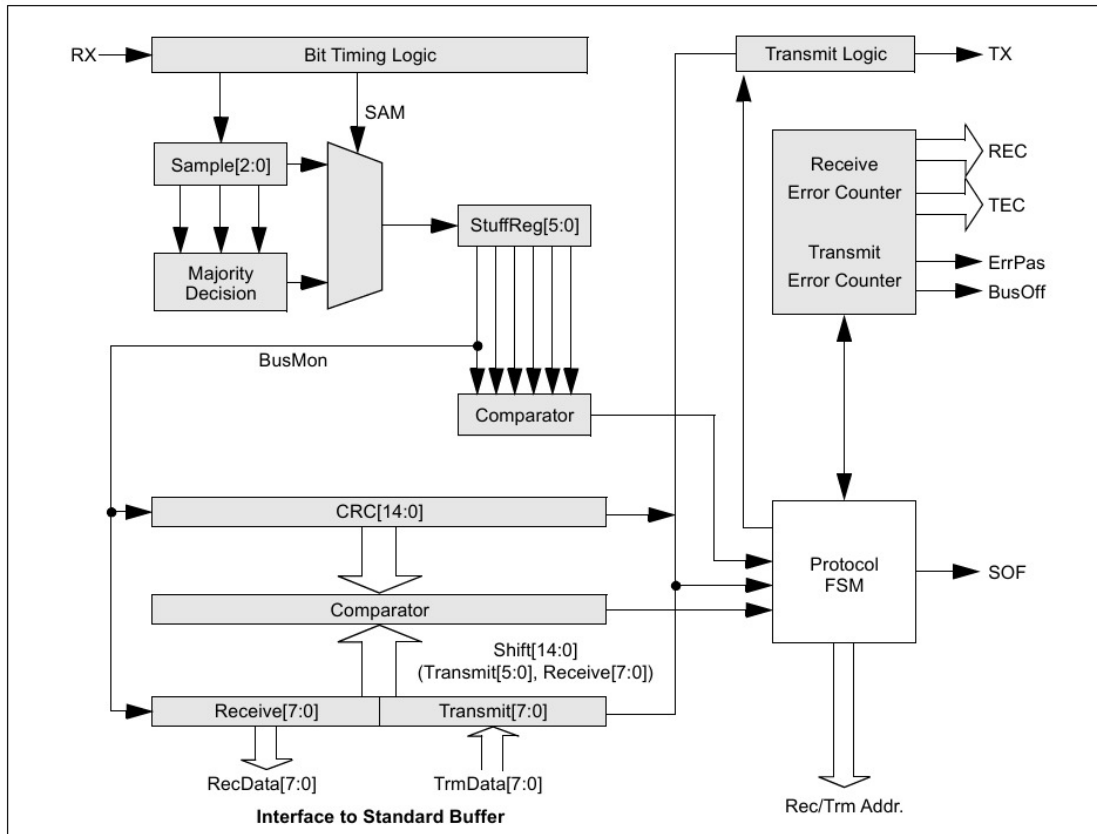


Figure 2: Reference internal block diagram of a CAN controller (bit-stream-processor architecture, e.g., similar to industry controllers such as the MCP2515), used to guide the RTL module partitioning in Phase 2.

opment board and interfaced with an appropriate CAN transceiver to establish a physical CAN bus (CAN_H/CAN_L), as illustrated in Figure 3. Multiple boards are subsequently configured as CAN nodes to demonstrate real communication between hardware-implemented controllers rather than simulated ones.

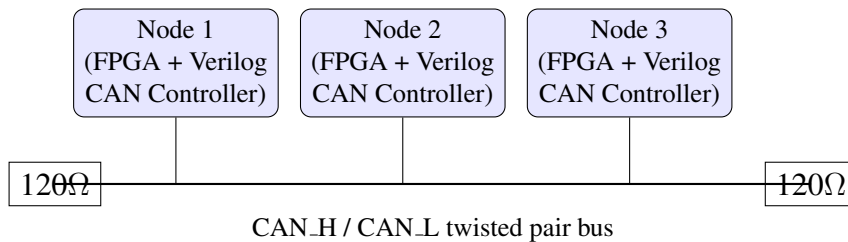


Figure 3: Multi-node CAN bus topology with FPGA-hosted Verilog CAN controllers and terminated ends.

7. Simulation, Verification & Reliability

Because the CAN controller in this project is built from scratch rather than reused from an existing IP block, its reliability guarantees have to be demonstrated in simulation before any

hardware bring-up is attempted, rather than assumed. The verification plan checks each of CAN's core reliability mechanisms as an explicit testbench scenario:

- **Bit monitoring:** verifying that a transmitting node correctly reads back the bus and detects a mismatch between the transmitted and monitored bit outside arbitration.
- **Bit stuffing and stuff-error detection:** verifying that a stuffed bit is correctly inserted/removed, and that a violation of the five-consecutive-identical-bits rule is correctly flagged.
- **CRC and form checks:** verifying that the receiver's independently computed CRC matches the transmitter's for valid frames, and correctly mismatches for deliberately corrupted frames.
- **Acknowledgement checking:** verifying that a transmitter correctly detects the absence of a dominant ACK bit when no simulated node is present to receive a frame.
- **Arbitration correctness:** verifying, with two or more simulated nodes transmitting simultaneously, that the node with the lower (higher-priority) identifier always wins the bus and that the losing node's transmission is not corrupted onto the bus.
- **Basic error scenarios:** deliberately injecting CRC mismatches, stuffing violations, and malformed frames in the testbench to confirm the receive controller flags them rather than silently accepting bad data.

Only after this simulation-level verification passes is the design targeted to hardware, where the same scenarios (particularly arbitration between real nodes and correct frame exchange over the physical bus) are re-checked to confirm that synthesis and the physical CAN transceiver layer preserve the behaviour verified in simulation.

8. Hardware and Software Requirements

Table 1: Indicative hardware and software requirements.

Category	Item
HDL & Simulation	Verilog HDL; simulator/waveform viewer (e.g., ModelSim/QuartaSim, Xilinx Vivado Simulator, or Icarus Verilog + GTKWave)
Synthesis Toolchain	FPGA vendor toolchain for synthesis and bitstream generation (e.g., Xilinx Vivado, Intel Quartus)
FPGA/Embedded Boards	3 or more FPGA/embedded development boards suitable for targeting the Verilog design (e.g., Xilinx/Intel FPGA dev boards)
CAN Transceivers	TJA1050 / SN65HVD230-class transceiver modules (one per node), to interface controller TX/RX to CAN_H/CAN_L

Category	Item
Bus Hardware	Twisted-pair cable, 120 Ω termination resistors, connectors/breadboard
Debug/Test Equipment	Logic analyzer or CAN bus analyzer/sniffer, digital multimeter, oscilloscope (for signal-level verification)

9. Tentative Timeline

Week(s)	Activity	Phase
1–2	Study of CAN architecture, frame structure, arbitration, and error handling	Phase 1
3–5	RTL design of transmit controller, receive controller, and bit-timing logic in Verilog	Phase 2
6–7	RTL design of arbitration, bit stuffing/de-stuffing, CRC, and ACK/error-detection logic	Phase 2
8–9	Integration of modules into a complete CAN controller	Phase 2
10–11	Testbench development and simulation-based verification (single- and multi-node scenarios)	Phase 3
12–13	FPGA/embedded synthesis, CAN transceiver interfacing, and hardware bring-up	Phase 4
14	Multi-node hardware demonstration, documentation, and final report	Phase 4

Table 2: Indicative project timeline.

10. Expected Outcomes

The project will result in the design and verification of a functional CAN protocol controller using Verilog HDL. The implementation demonstrates the core mechanisms of CAN at the RTL level rather than treating CAN merely as an existing peripheral. Specifically, by the end of the project the following outcomes are expected:

- A complete, simulation-verified Verilog RTL design of a CAN controller, covering frame generation, frame reception, bit-level arbitration, bit stuffing/de-stuffing, CRC generation and checking, and ACK/error handling.
- A Verilog testbench suite demonstrating correct multi-node arbitration, error detection, and successful communication between simulated CAN nodes, with waveform-level evidence of correct FSM and signal behaviour.
- A working FPGA/embedded hardware implementation of the controller, interfaced with a CAN transceiver, demonstrating real communication between multiple hardware CAN nodes on a physical bus.

- Practical experience in digital system design, Verilog HDL, finite-state machines, serial communication protocol design, error detection, and embedded hardware interfacing – following the progression CAN Protocol Study → RTL Architecture → Verilog Implementation → Simulation and Verification → FPGA/Embedded Implementation → CAN Network Demonstration.

11. Applications (Illustrative, Optional Extension)

The scope of this project (Section 5) is limited to the design, verification, and hardware realization of the CAN controller itself; no application layer is a mandatory deliverable. This section is included only to illustrate one possible domain the finished controller could be demonstrated in, as a direction for future work if time permits after the core RTL and hardware objectives (Phases 1–4) are met. The specific functions listed below are indicative examples, not a committed feature list, and may be changed, reduced, or replaced entirely depending on time available.

A natural illustrative application area for the CAN controller designed in this project is a small-scale automotive control system, used here only to show how the underlying protocol could map onto real ECU-to-ECU communication in a vehicle. One possible idea is a miniature network of Electronic Control Units (ECUs) that exchange data over CAN and coordinate simple vehicle-like functions, without requiring an actual car or automotive-grade hardware. Example functions that *could* be explored, time permitting, include:

- **Adaptive Cruise Control (illustrative):** a simulated sensor node sends speed/distance data over CAN; a controller node processes this and issues commands to a motor/brake node.
- **Driving-Mode-Based Adaptive Suspension (illustrative):** a selected mode (e.g., Comfort/Normal/Sport) is transmitted over CAN to a suspension node, which adjusts a simulated damping parameter accordingly.
- **Vehicle Parameter Monitoring (illustrative):** sensor nodes transmit parameters such as speed, temperature, acceleration, or battery voltage over the bus for display/logging.
- **Basic Fault Detection (illustrative):** CAN traffic is monitored for abnormal values or missing messages, to flag possible node or link faults.

Figure 4 sketches one possible high-level architecture for such a demonstration, purely to illustrate the idea; the exact nodes and functions, if pursued, would be finalized later based on time and hardware available.

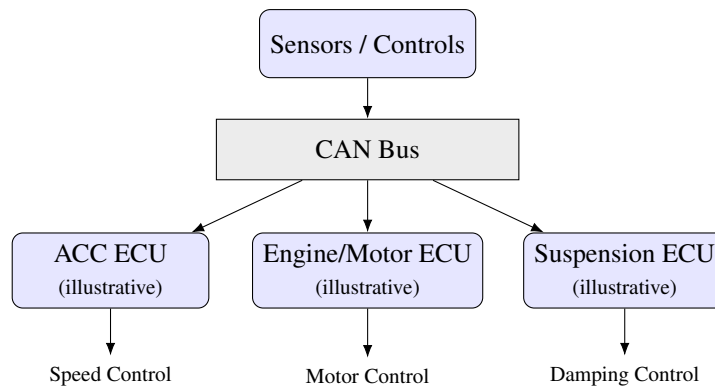


Figure 4: One possible illustrative application architecture built on the designed CAN controller (indicative only; not a committed project deliverable).

12. References

1. Robert Bosch GmbH, *CAN Specification, Version 2.0*, 1991.
2. ISO 11898-1:2015, *Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling*, International Organization for Standardization.
3. NXP Semiconductors, *SJA1000 Stand-alone CAN Controller – Datasheet*. [Online]. Available: <https://www.nxp.com>
4. Microchip Technology Inc., *MCP2515 Stand-Alone CAN Controller with SPI Interface – Datasheet*. [Online]. Available: <https://www.microchip.com/en-us/product/MCP2515>
5. Texas Instruments, *Introduction to the Controller Area Network (CAN)*, Application Report SLOA101, 2016. [Online]. Available: <https://www.ti.com/lit/an/sloa101b/sloa101b.pdf>
6. W. Voss, *A Comprehensible Guide to Controller Area Network*, 2nd ed. Copperhill Media, 2008.
7. S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.